

Building a TJHSST ADAPT-VQE

Justin Lee,¹ William Black,¹ Marina Lin,¹ Japneet Kaur,¹ Willie Henry,¹ Rabia Chadha,¹ Akansha Bagga,¹ Kade Yen,¹ and Sophia Lin¹

Thomas Jefferson High School for Science and Technology

(Dated: 3 June 2025)

We implemented TJHSST’s version of the ADAPT-VQE (Adaptive Derivative-Assembled Pseudo-Trotter Variational Quantum Eigensolver) algorithm. We benchmark the performance across various observable metrics, operator pools, and optimization strategies, especially in the presence of sampling noise for quantum chemistry.

I. INTRODUCTION

Quantum computing promises to revolutionize computations ranging from financial analyses to breaking modern cryptography. Quantum computers are here today, but do not yet have the power of ideal quantum computers; they are generally small and/or noisy, which limits their utility in the near term. As such, we are said to be in the Noisy Intermediate-Scale Quantum (NISQ) era¹. There are a number of extraordinary algorithms that require a error-corrected quantum computer to be useful. However, in the NISQ era, we can still design algorithms that are able to use quantum computers despite their noise.

Variational Quantum Eigensolvers (VQEs) are hybrid quantum-classical algorithms designed to approximate the ground state and ground state energy of a given Hamiltonian². It allows for predictions of bonding and applications to medicine³ and material science⁴ such as drug and vaccine discovery³. In molecular computations, classical methods often struggle with harsh scaling⁵. The benefit of VQE and a quantum processor is that one may efficiently store the exponentially-scaling size of their quantum states [note: this exponential scaling is different from the "harsh scaling" in the previous sentence, which does not necessarily refer to the issues of storing the wavefunction, though it’s a necessary part of it]. There have been numerous studies on physical implementations of VQE on current quantum computers⁶. VQEs also have the benefit of being variational, meaning that calculated energy values may not be less than the ground state energy.

This variational approach is based on an *ansatz*, a parameterized quantum circuit that approximates the ground-state wave function. By varying the parameters of the *ansatz*, the algorithm seeks to minimize the expectation value of the Hamiltonian. There are numerous choices of *ansätze*; for example, a chemistry-inspired *ansatz* may match molecular structure, like the unitary coupled cluster (UCC) *ansatz*⁷, or a hardware-efficient *ansatz*⁸ may be based on what a given quantum computer can run on its architecture.^w

The Adaptive Derivative-Assembled Pseudo-Trotter Variational Quantum Eigensolver (ADAPT-VQE)⁹ builds on VQEs by dynamically growing the *ansatz*, operator by operator, re-optimizing the parameters each time. This avoids unnecessary parameters and circuit depth. Operators are selected from a set of operators, called the *pool*, by taking the derivatives of the energy with respect to their parameters. The operator with the highest gradient is selected to be added to the *ansatz*. There

exist a number of variations upon this theme, such as TETRIS-ADAPT-VQE, which fills as many operators as possible into a layer in order of each operator with the largest gradient¹⁰.

Our research was conducted by nine members split into three groups, each with a specific research question and focus. The first group, which consists of Akansha Bagga, Marina Lin, Sophia Lin, and Kade Yen, studied the importance of classical optimizers in VQE, as well as how they are affected by sampling noise. The second group, which studied observables and preservation of symmetries, consisted of Rabia Chadha and Japneet Kaur. The third group, composed of William Black, Willie Henry, and Justin Lee, explored the tiled Unitary Product State *ansatz*¹¹ as well as a number of pools deriving from the tUPS *ansatz*. Each group contributed to the design and development of a codebase implementing VQE, ADAPT-VQE, as well as the necessary associated pools and optimizers¹²

II. METHODS

Libraries Used

To construct and simulate the variational quantum eigensolver (VQE), we employed a suite of specialized Python libraries. For handling molecular structures and second-quantized operators, we used `openfermion` and `openfermionpyscf`. Quantum circuit construction and execution were facilitated by Qiskit’s modular stack, including `qiskit`, `qiskit_aer` for simulation backends, `qiskit_algorithms` for high-level algorithmic interfaces, and `qiskit_ibm_runtime` to execute circuits on IBM’s cloud-based quantum systems. Classical optimization tasks were handled using `scipy`, and experiment tracking and result visualization were managed through `mlflow` and `matplotlib`, respectively.

The variational quantum eigensolver was constructed using the Qiskit framework in conjunction with simulators provided by `qiskit_aer`. The quantum state is represented by a parameterized `QuantumCircuit` object in Qiskit, which encodes the trial wavefunction. Expectation values of the Hamiltonian are estimated through repeated sampling of the circuit. The optimization process to minimize the energy is executed classically using a variety of techniques, including both gradient-based and gradient-free approaches, thereby leveraging the hybrid quantum-classical nature of the VQE algorithm.

The observables used in the VQE algorithm were con-

structured using the OpenFermion library. These observables are specified in terms of creation and annihilation operators acting on qubits. To translate these into quantum circuits, we applied the Jordan–Wigner transformation, which maps fermionic operators to Pauli strings. The resulting pool of operators is then added to the quantum circuit. Each operator is parameterized and exponentiated in the form $\exp(A \cdot \theta)$, where A is the operator and θ is the variational parameter.

To support flexible optimization within the VQE framework, we implemented a modular hierarchy of optimizers that includes both gradient-based and gradient-free techniques. The `scipy` library’s optimization routines were used to iteratively minimize the expected energy with respect to the current ansatz parameters. These routines return updated parameters for successive energy evaluations. Among the implemented methods are stochastic gradient descent, adaptive movement estimation, trust-region strategies, COBYLA, and BFGS, with BFGS set as the default optimizer which are defined more clearly in Section III A. This flexible setup allows tailoring the optimization strategy to the specific characteristics of the problem.

III. OPTIMIZERS

In this section, we investigate how sampling noise influences the performance of ADAPT-VQE when paired with different classical optimization methods. Sampling noise arises from the finite number of measurements used to estimate expectation values on quantum circuits, potentially impacting the convergence and stability of the optimization process. To assess this, we compare several optimizers: Stochastic Gradient Descent (SGD), ADAM, Broyden–Fletcher–Goldfarb–Shanno (BFGS), Trust Region, and Constrained Optimization by Linear Approximation (COBYLA). Our goal is to identify which algorithms are more robust to sampling fluctuations and maintain effective convergence under noise. We used 2^{20} shots or measurements and the standard deviation associated due to sampling noise is ± 0.001 .

A. Description of Optimizers

1. Stochastic Gradient Descent (SGD)

As a baseline, we use SGD¹³ as a simple, gradient-based method that updates the parameters one step at a time using noisy gradient estimates. Each training sample is passed through the neural network one at a time, and parameters are updated with the computed gradient. An example of an update step is shown in Figure 1 and the update rule below.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})$$

where θ_t is the current parameter vector at iteration t , η is the learning rate, which controls the step size of the update,

$\nabla_{\theta} \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})$ is the gradient of the loss function $\mathcal{L}$ with respect to the parameters θ , computed on a single training example $(x^{(i)}, y^{(i)})$, θ_{t+1} is the updated parameter vector after taking a step in the direction of the negative gradient.

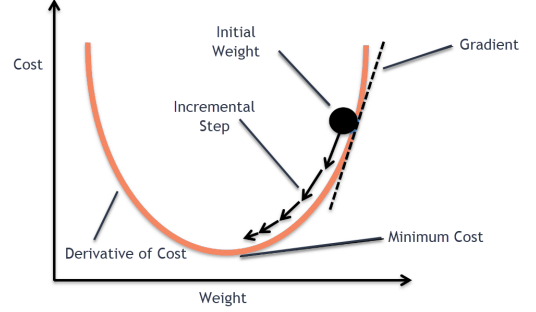


FIG. 1. SGD¹⁴

2. Adaptive Moment Estimation (ADAM)

ADAM is a gradient-based optimization algorithm that combines the benefits of two key ideas: momentum and adaptive learning rates. Momentum involves maintaining an exponentially decaying average of past gradients, which helps smooth out noisy updates and accelerates convergence, particularly in the presence of high variance in the gradient estimates. In parallel, ADAM incorporates techniques from RMSProp by using an exponentially decaying average of the squared gradients to scale learning rates adaptively for each parameter. This ensures that each parameter has a customized step size, preventing issues like overshooting or stagnation. In particular, there is more update in parameters based on both the magnitude and direction of past gradients, enabling more stable and efficient traversal of the loss surface, especially in noisy or non-convex optimization landscapes. As a result, we think it makes it particularly well-suited for VQE where sampling noise is a major concern.

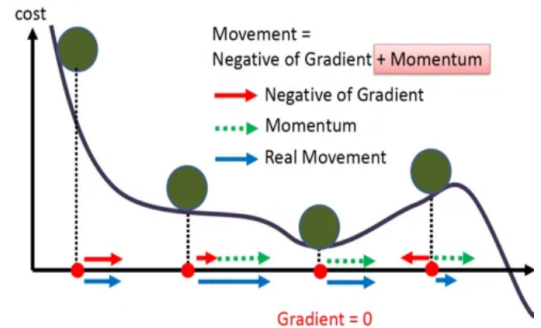


FIG. 2. ADAM Momentum¹⁵

3. Broyden–Fletcher–Goldfarb–Shanno (BFGS)

The Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm¹⁶ is a widely used quasi-Newton optimization

method that avoids the computational expense of explicitly computing second derivatives. Instead, it builds an approximation to the inverse Hessian matrix using only gradient information. At each iteration, the algorithm computes the search direction

$$\mathbf{p}_k = -\mathbf{H}_k \nabla f_k,$$

where $\mathbf{H}_k$ is the current estimate of the inverse Hessian matrix and ∇f_k is the gradient of the objective function at iteration k . A line search is performed to find a step size α_k satisfying the Wolfe conditions, and the parameters are updated as

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k.$$

To update the Hessian approximation, BFGS uses the differences in positions and gradients between successive iterations:

$$\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad \mathbf{y}_k = \nabla f_{k+1} - \nabla f_k.$$

Using these, the inverse Hessian approximation is updated via the formula

$$\mathbf{H}_{k+1} = \left(\mathbf{I} - \frac{\mathbf{s}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} \right) \mathbf{H}_k \left(\mathbf{I} - \frac{\mathbf{y}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k}.$$

This approach provides search directions that are often superior to those of simple gradient descent—steeper and improves convergence speed. Due to its balance between computational cost and curvature awareness, BFGS is particularly prevalent in Variational Quantum Eigensolver (VQE) problems, where full second-order methods are typically too expensive.

4. Trust Region Method

Trust region methods are a class of second-order optimization algorithms that focus on building reliable local approximations of the objective function. Instead of taking potentially risky global steps, these methods adopt a conservative strategy by restricting updates to lie within a region known as the *trust region* around the current iterate where the model is assumed to be a good approximation. This region is typically defined as a ball around the current point $\mathbf{x}_k$, such that the step $\mathbf{p}$ satisfies $\|\mathbf{p}\| \leq \Delta_k$, where Δ_k is the trust region radius at iteration k .

The trust region subproblem aims to minimize a quadratic approximation of the objective function within this region:

$$\min_{\mathbf{p}} \quad m_k(\mathbf{p}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \mathbf{B}_k \mathbf{p},$$

subject to $\|\mathbf{p}\| \leq \Delta_k$ where $\mathbf{B}_k$ is typically the Hessian or its approximation. After solving the subproblem, the actual reduction $f(\mathbf{x}_k) - f(\mathbf{x}_k + \mathbf{p})$ is compared to the predicted reduction $m_k(\mathbf{0}) - m_k(\mathbf{p})$. This comparison informs whether the step was successful and guides the adjustment of the trust region size Δ_k . If the model predicted the improvement well, the region may be expanded; otherwise, it is contracted.

Trust region methods are particularly effective in noisy or uncertain environments, as they manage uncertainty by restricting updates to reliable neighborhoods. They also adapt the step size dynamically, ensuring stable updates even under sampling noise. These properties make trust region approaches robust for problems where global approximations or large parameter updates are unreliable.

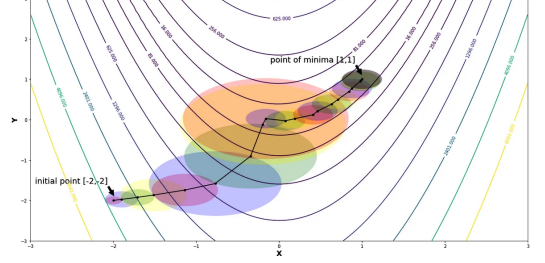


FIG. 3. Trust Region Visual¹⁷

5. Constrained Optimization by Linear Approximation (COBYLA)

COBYLA is a derivative-free optimization method designed for problems where gradients are unavailable or unreliable. It constructs linear approximations to both the objective and constraint functions, enabling optimization without requiring explicit derivative information. At each iteration, COBYLA approximates the nonlinear objective $f(\mathbf{x})$ and any constraint functions $g_i(\mathbf{x})$ using linear models based on interpolation from a set of points in the parameter space. The algorithm then solves a trust-region subproblem in a linear space:

$$\min_{\mathbf{p}} \quad f(\mathbf{x}_k) + \mathbf{c}^\top \mathbf{p} \quad \text{subject to} \quad \|\mathbf{p}\| \leq \Delta_k, \quad \mathbf{A}\mathbf{p} \leq \mathbf{b},$$

where $\mathbf{c}$ approximates the gradient direction, Δ_k defines the trust region radius, and $\mathbf{A}$ and $\mathbf{b}$ encode the linear constraints. Unlike traditional trust-region methods, which rely on quadratic models and second-order information, COBYLA operates purely in a linear regime, making it well-suited for problems with noisy or discontinuous objectives where derivative estimates are too costly.

B. Results and Discussion

1. Single ADAPT Iteration

In both the noise-free and noisy settings, we observed consistent trends across optimizers, with similar behavior replicated when switching from the GSD pool to the QEB pool. Figure 4 illustrates the performance of various optimizers under ideal, noise-free conditions. In this setting, SGD demonstrates very slow convergence—it gradually reduces energy error but fails to achieve high precision within a reasonable number of iterations. Adam performs moderately better, achieving a reasonable energy drop, though it still lags behind

second-order methods. COBYLA initially shows progress but lacks stability and fails to converge to low error. L-BFGS has extremely fast and deep convergence, reaching near-optimal energy in just a few iterations. Trust Region optimization also converging rapidly and reaching the lowest energy error, indicating both fast and precise behavior.

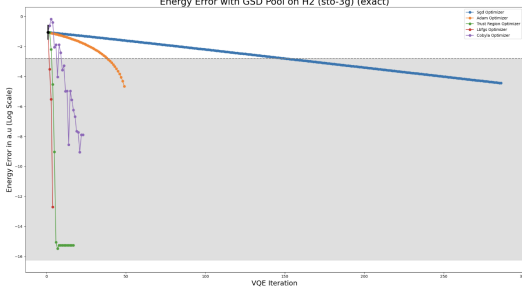


FIG. 4. Single ADAPT Iteration with No Sampling Noise

Figure 5 presents the results under sampling noise. Here, SGD maintains its characteristic stability but is extremely slow—it does not reach low energy within the given iteration window. Adam still outperforms SGD and converges faster, but its effectiveness is limited by the presence of noise. COBYLA becomes highly unstable, without meaningful convergence. L-BFGS initially shows a sharp drop in energy error but quickly becomes unstable. In contrast, the Trust Region method maintains a relatively smooth and reliable convergence trajectory. Although its descent is not as steep as in the noise-free case, it remains the most resilient to gradient noise and consistently reaches low energy levels. Thus, under noisy conditions, Trust Region stands out as the most effective optimizer, combining robustness, stability, and dependable convergence.

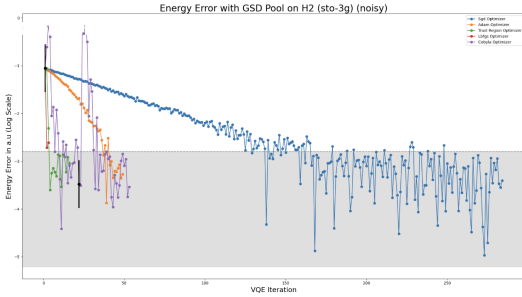


FIG. 5. Single ADAPT Iteration with Sampling Noise

2. Multiple ADAPT Iterations

Extending our findings from a single ADAPT iteration, we observe that L-BFGS consistently delivers the best performance across multiple ADAPT iterations. This robustness stems from its ability to combine curvature information with a line search, enabling fast yet stable convergence to low energy states. Unlike first-order methods such as SGD, which

rely solely on a single gradient direction and fixed or heuristic step sizes, L-BFGS adapts its step based on both past gradients and how the objective function is changing. This allows it to take larger and more confident steps in well-behaved regions. This trend can be seen in green line in Figure ?? and Figure 7.

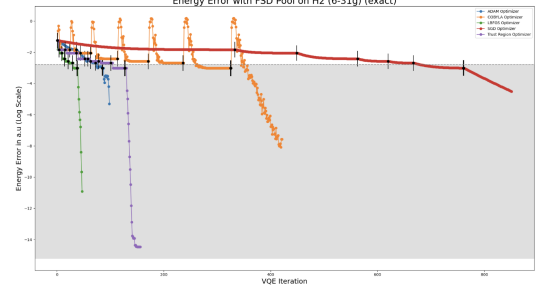


FIG. 6. Multiple ADAPT Iteration with Sampling Noise

On the other hand, COBYLA performs poorly in this context due to its reliance on simplistic local linear approximations. Because it perturbs one parameter at a time and assumes that parameters affect the cost independently, COBYLA fails to capture the curvature and interactions between parameters. This leads to erratic behavior and instability, especially under noise. Trust Region methods, while slower, offer smoother and more reliable convergence in both exact and noisy conditions by shrinking the step size when uncertainty is high, thus avoiding overshooting or noisy jumps.

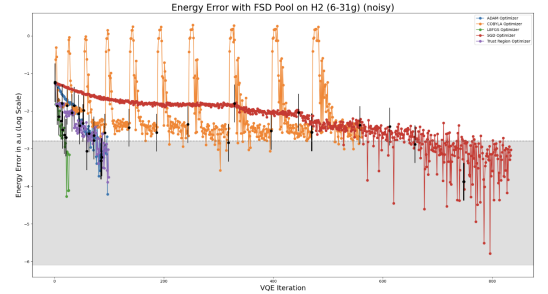


FIG. 7. Trust Region

Taken together, our results show that while simpler optimizers may occasionally perform well in ideal conditions, second-order methods like L-BFGS and potentially Trust Region and Adam scale more effectively with ADAPT's iterative nature under noise conditions. As the circuit grows more complex across iterations, optimizers that can account for curvature and adjust step sizes dynamically are better equipped to sustain both speed and accuracy.

IV. OBSERVABLES

In this section, we explore how fermionic quantities change along the evolution of the ADAPT-VQE algorithm. We examined three different fermionic quantities: 1) Particle number

(total number of fermions in the system), 2) Spin z projection (projection of the particle's spin in the z-direction), and 3) Total spin projection (sum of the projection of the particle's spin in all three directions squared). These observables are calculated by taking the expectation value of the operator associated with each variable with the reference state (ψ) produced at the end of a run of the ADAPT-VQE algorithm ($\langle \psi | n | \psi \rangle$, $\langle \psi | S_z | \psi \rangle$, $\langle \psi | S^2 | \psi \rangle$, where n , S_z , and S^2 are the operators associated with each fermionic quantity). We also aimed to develop a dynamic ADAPT-VQE, where instead of selecting operators from one pool, we would switch to a different pool entirely during the course of the algorithm based on various factors, including energy convergence and CNOT depth.

A. Tracking Fermionic Quantities

Maintaining symmetry is vital for the ADAPT-VQE in improving computational efficiency and ensuring simulated states to be physically meaningful. When simulating the lithium hydride molecule with a typical bond length of 2.0 Å, the fermionic quantities of the molecule remained conserved in the qubit excitation pool. The qubit excitation (QE) pool is known to conserve symmetries of common fermionic observables like the three mentioned above. However, in the No Pauli-Z pool, the number observable was not conserved by a minimal amount, a known characteristic of this pool. The No Pauli-Z pool is a subset of the Qubit pool operators (used in our simulation of the dynamic pool below), as it is constructed in a similar way to the Qubit pool, having eliminated the Z Pauli strings.

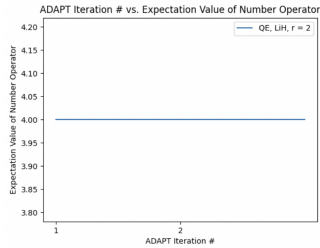


FIG. 8. Number Observable Using QE Pool

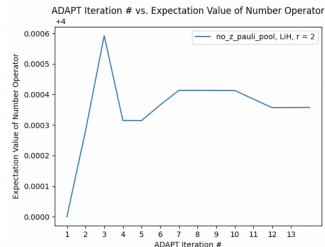


FIG. 9. Number Observable Using No Pauli Z Pool

B. Results and Discussion: Developing a Dynamic Pool

To determine if the dynamic pool was a valuable idea, we formulated and constructed a combined pool made up of QE and Qubit pool operators. We ran the ADAPT-VQE algorithm with three pools: QE (referred to as QEB in the figures below), Qubit, and a combined pool, with both the QE and Qubit operators. We chose these pools as since the Qubit operators are linear combinations of the QE pool, so the gradients can be calculated faster with the correct implementation. In tracking how well each pool of the three levels conserved our observables, we simulated the molecule H_4 , a theoretical molecule using a 0.5 Å bond length.

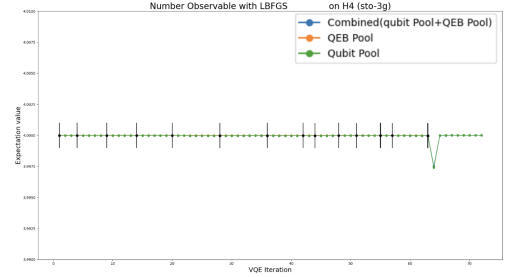


FIG. 10. Number Observable

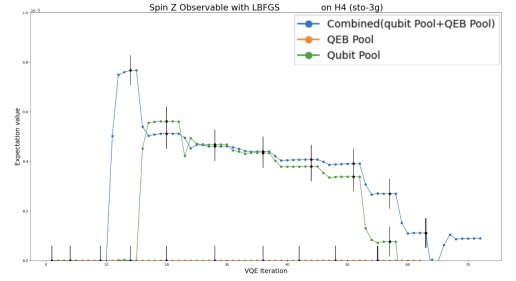


FIG. 11. Spin Z Observable

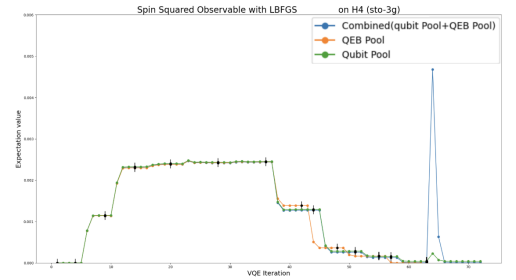


FIG. 12. Spin Squared Observable

The QE pool was able to preserve the first two observables, particle number and S_z because there are no fluctuations in either observable at each ADAPT iteration. The expectation value of the total S^2 observable of the QE pool did change per algorithm iteration, however. Yet, this change reaches it

maximum at 0.003, which is minimal. Thus, the total S^2 observable changing may indicate a potential error in our pool implementation as QE must preserve all symmetries or it may be error in our logging. The combined pool conserved particle number as well, but did not conserve the S_z observable. In regards to S^2 , we consider it not to be conserved due to the large spike at ADAPT iteration 12 (indicated by the black mark) and around VQE iteration 65. The absence of symmetry conservation was due to the fact that this pool contained operators from the Qubit pool, which was shown not to conserve symmetries like the No Pauli-Z pool. Thus, creating a dynamic ADAPT-VQE may not provide a significant advantage in the conservation of fermionic quantities. However, this must be further researched as the operator gradients were not conducted faster due to combined pool consisting of an addition of operator arrays. Thus, the experiment did not improve efficiency or time convergence, which must be further tested. Moreover, additional molecules with varying bond lengths should be simulated for further trials.

V. POOLS

The tiled Unitary Product State (tUPS) ansatz relies on the following key observation: any quantum state may be written as

$$|\Psi(\theta)\rangle = \prod_{j=1}^M \exp(\theta_j \hat{\kappa}_{\mu_j}) |\Phi_0\rangle \quad (1)$$

for some reference state $|\Phi_0\rangle$, some arbitrary vector of parameters θ , and one and two body operators between arbitrary orbitals $\hat{\kappa}_{\mu_j}$. This is called the unitary product state (UPS). As the author notes, this form may be further simplified to only include the spin-adapted one body and paired two body operators $\hat{\kappa}_{pq}^{(1)}$ and $\hat{\kappa}_{pq}^{(2)}$ which act between spatial orbitals p and q , defined by

$$\hat{E}_{pq} = \hat{a}_p^\dagger \hat{a}_q + \hat{b}_p^\dagger \hat{b}_q \quad (2)$$

$$\hat{\kappa}_{pq}^{(1)} = \hat{E}_{pq} - \hat{E}_{qp} \quad (3)$$

$$\hat{\kappa}_{pq}^{(2)} = \hat{E}_{pq}^2 - \hat{E}_{qp}^2, \quad (4)$$

for annihilation operators a_p and b_p , which respectively annihilate a spin-down and a spin-up electron in the p orbital. In principle, 1 can be used to represent any quantum state as $M \rightarrow \infty$, but in practice, we use a finite M . The author proposes the tUPS ansatz

$$|\Psi_{tUPS}\rangle = \prod_{m=1}^L \left[\prod_p \hat{U}_{2p+1,2p}^{(m)} \prod_p \hat{U}_{2p,2p-1}^{(m)} \right] |\Phi_0\rangle, \quad (5)$$

where the operators are defined by

$$\hat{U}_{pq}^{(m)} = \exp(\theta_{pq,1}^{(m)} \hat{\kappa}_{pq}^{(1)}) \exp(\theta_{pq,2}^{(m)} \hat{\kappa}_{pq}^{(2)}) \exp(\theta_{pq,3}^{(m)} \hat{\kappa}_{pq}^{(1)}). \quad (6)$$

The tUPS ansatz restricts the form of the UPS expression to interactions between adjacent spatial orbitals, and then interleaves the operators in layers.

We compare the pools developed from the tUPS ansatz with standard pools, such as the pool of restricted fermionic excitations (hereinafter FSD)⁹, the pool generalized fermionic excitations (GSD)⁹, and the pool of qubit excitations (QEB)¹⁸.

A. tUPS Pools

We present four pools based off the tUPS ansatz. The first of the four pools, called the *full tUPS pool*, was simply one operator, which was one layer of the tUPS ansatz. In other words, the operator

$$\prod_p \hat{U}_{2p+1,2p}^{(m)} \prod_p \hat{U}_{2p,2p-1}^{(m)}. \quad (7)$$

The possible use case for this is that one may adaptively determine how long the tUPS ansatz circuit needs to be. Otherwise one would need to estimate how long the circuit should be, and would either not get enough depth to accurately represent the ground state, or has too many layers and too many parameters to easily optimize the ground state.

The second pool we considered, called the *individual tUPS pool* was the set of all exponentiated $\hat{\kappa}_{pq}^{(1)}$ and $\hat{\kappa}_{pq}^{(2)}$ operators.

The third and fourth pools both consisted of the $U_{pq}^{(m)}$ operators. The third pool, called the *adjacent tUPS pool*, contained operators only between adjacent spatial orbitals. In contrast, the fourth pool, which is named the *multi tUPS pool*, allowed operators between any two spatial orbitals.

B. Results and Discussion

Multiple runs of ADAPT-VQE and VQE using various pools and Ansätze can be seen in the following figures.[?] shows a collection of computations using the STO-3G basis set with the COBYLA optimizer, and[?] displays the same for the 6-31G basis set but with LBFGS as the classical optimizer.

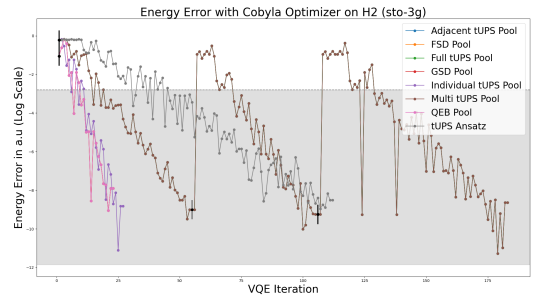


FIG. 13. H₂ runs using STO-3G

With the STO-3G basis, the three standard pools (FSD, GSD, and QEB) converge relatively quickly (simultaneously on the graph), each taking about 45 iterations. The individual tUPS pools converge at a rate to the standard pools. For H₂ with the STO-3G basis, there are only two spatial orbitals, so the Multi and Adjacent tUPS-pools produce the same exact

results, reaching chemical accuracy in more iterations than the standard pools, but with less than the tUPS Ansatz.

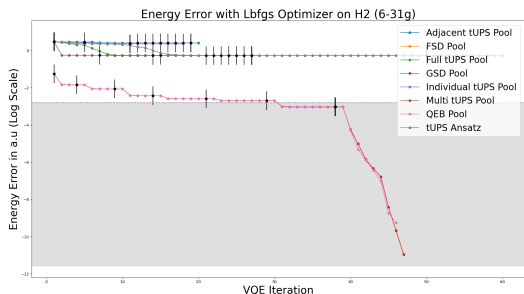


FIG. 14. H_2 runs using 6-31G

Under the 6-31G basis set, only the three standard pools converge, all relatively quickly and at a similar pace (note that FSD and GSD are simultaneous on the graph). None of the tUPS pools, nor the tUPS ansatz itself converge. The individual and adjacent tUPS pools both barely lower the energy from the reference state, while the full tUPS pool, Multi tUPS pool, and the tUPS ansatz converge at a slightly lower energy value than the Hartree Fock energy. The tUPS ansatz itself should be able to converge for a small molecule like H_2 , and at the very least, the full tUPS pool should also be able to converge. The issues with the tUPS ansatz and pools not converging can likely be attributed to the difficulty of optimizing the tUPS ansatz. Just each operator in the Ansatz contains three parameters, whereas in pools like FSD or QEB, there is only one parameter per operator. Furthermore, each layer of the tUPS ansatz contains approximately as many of these operators as the number of spatial orbitals. With a better method of optimizing tUPS ansatzes in general, these pools may be able to converge.

VI. CONCLUSION

We successfully implemented a TJHSST version of ADAPT-VQE. Our implementation can support various pool designs, adaptive or static ansatzes, multiple optimizer paradigms, and the conservation of arbitrary observables. We used this implementation to run experiments on various molecules with different combinations of optimizers and pools. For each small group, our conclusions are as follows:

1. Sampling Noise: Optimizer are affected to varying degrees by the introduction of sampling noise. L-BFGS, Trust Region, and Adam seems to be the most stable with sampling noise due to their ability to adjust step size.
2. Fermionic Quantities: We observed that the QE pools did conserve the particle number and spin z observable, but not spin squared. Our attempts to create a dynamic pool similarly resulted in a pool that only conserved the particle number and spin z observable.

3. Pools: We created four distinct pools based on the tUPS ansatz. These ansatzes were able to converge to chemical accuracy on the H_2 molecule with the STO-3G basis set. However, when testing on H_2 with the 6-31G basis set, all the tUPS based ansatzes failed to converge. The FSD, GSD, and QEB pools all converged in these instances.

A. Future Work

Given the time constraints of the project, our implementation might not be perfect. For example, the UCC ansatzes should converge on H_2 , but do not. Future work should go into testing each element of our VQE to make sure it works properly. As for future directions for each group, they are as follows:

1. Sampling Noise: Given our results, we want to investigate how we can improve optimizers such as L-BFGS, Adam, and Trust Region that perform well in exact conditions but struggle with the introduction of sampling noise.
2. Fermionic Quantities: We want to see how other pools, different molecules, and different bond distances, affect the algorithm's ability to preserve fermionic quantities. We also want to see if dynamic pools that do conserve these quantities are feasible.
3. Pools: Try and design other pools that are based off of the tUPS ansatz. Come up with an effective way to optimize tUPS based ansatzes, both static and adaptive, to converge more consistently to chemical accuracy.

ACKNOWLEDGMENTS

We would like to to acknowledge the support of our lab director, Mr. Mark Hannum. We are also indebted to our mentors at Virginia Tech, namely, Dr. Kyle Sherbert (optimizers group), Dr. Karunya Shirali (observables group), and Dr. Mafalda Ramôa (pools group), as well as lab director Dr. Sophia Economou.

REFERENCES

- ¹J. Preskill, "Quantum computing in the nisq era and beyond," *Quantum* **2**, 79 (2018).
- ²A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications* (2014).
- ³Y. Cao, J. Romero, and A. Aspuru-Guzik, "Potential of quantum computing for drug discovery," *IBM Journal of Research and Development* **62**, 6:1–6:20 (2018).
- ⁴V. Lordi and J. M. Nichol, en"Advances and opportunities in materials science for scalable quantum computing," *MRS Bull.* **46**, 589–595 (2021).
- ⁵J. D. Whitfield, P. J. Love, and A. Aspuru-Guzik, "Computational complexity in electronic structure," *Phys. Chem. Chem. Phys.* **15**, 397–411 (2013).

- ⁶L. Zhao, J. Goings, K. Shin, W. Kyoung, J. I. Fuks, J.-K. Kevin Rhee, Y. M. Rhee, K. Wright, J. Nguyen, J. Kim, and S. Johri, “Orbital-optimized pair-correlated electron simulations on trapped-ion quantum computers,” *npj Quantum Information* **9** (2023), 10.1038/s41534-023-00730-8.
- ⁷A. Anand, P. Schleich, S. Alperin-Lea, P. W. K. Jensen, S. Sim, M. Díaz-Tinoco, J. S. Kottmann, M. Degroote, A. F. Izmaylov, and A. Aspuru-Guzik, “A quantum computing view on unitary coupled cluster theory,” *Chemical Society Reviews* (2022).
- ⁸L. Leone, S. F. Oliviero, L. Cincio, and M. Cerezo, “On the practical usefulness of the hardware efficient ansatz,” *Quantum* **8**, 1395 (2024).
- ⁹H. R. Grimsley, S. E. Economou, E. Barnes, and N. J. Mayhall, “An adaptive variational algorithm for exact molecular simulations on a quantum computer,” *Mature Communications* (2019).
- ¹⁰P. G. Anastasiou, Y. Chen, N. J. Mayhall, E. Barnes, and S. E. Economou, “Tetris-adapt-vqe: An adaptive algorithm that yields shallower, denser circuit ansätze,” *Physical Review Research* (2024).
- ¹¹H. G. A. Burton, “Accurate and gate-efficient quantum ansätze for electronic states without adaptive optimization,” *Physical Review Research* (2024).
- ¹²J. Lee, W. Black, M. Lin, J. Kaur, W. Henry, R. Chadha, A. Bagga, K. Yen, and S. Lin, “Tj-adapt-vqe,” <https://github.com/NotFish232/TJ-ADAPT-VQE> (2025), accessed: 2025-06-02.
- ¹³S. ichi Amari, “Backpropagation and stochastic gradient descent method,” *Neurocomputing* **5**, 185–196 (1993).
- ¹⁴Divakar, “Stochastic vs batch gradient descent,” https://medium.com/@divakar_239/stochastic-vs-batch-gradient-descent-8820568eada1 (2018), accessed: 2025-05-29.
- ¹⁵A. Bajaj, “Momentum, rmsprop and adam optimizer,” <https://medium.com/analytics-vidhya/momentum-rmsprop-and-adam-optimizer-5769721b4b19> (2020), accessed: 2025-05-29.
- ¹⁶J. W. Mullinax, P. G. Anastasiou, J. Larson, S. E. Economou, and N. M. Tubman, “Classical preoptimization approach for adapt-vqe: Maximizing the potential of high-performance computing resources to improve quantum simulation of chemical applications,” *Journal of Chemical Theory and Computation* **21**, 4006–4015 (2025).
- ¹⁷S. Khare and A. Kumar, “Trust region methods,” <https://medium.com/trust-region-methods/temp-blog-1b51189594a> (2021), published on Medium, Jan 17, 2021. 7 min read.
- ¹⁸Y. S. Yordanov, V. Armaos, C. H. W. Barnes, and D. R. M. Arvidsson-Shukur, “Qubit-excitation-based adaptive variational quantum eigensolver,” *Communications Physics* **4** (2021), 10.1038/s42005-021-00730-0.